Module 4:
Advanced Graph Algorithms
(Part-1)

Dr. Nirnay Ghosh

Graph Coloring

- Definitions:

A *coloring* of a simple graph is the assignment of a color to each vertex of the graph so that no two adjacent vertices are assigned the same color.

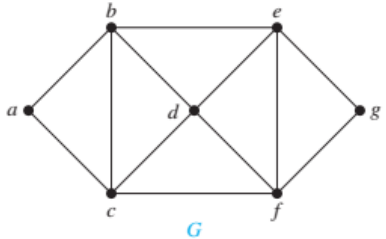
The *chromatic number* of a graph is the least number of colors needed for a coloring of this graph. The chromatic number of a graph G is denoted by $\chi(G)$. (Here χ is the Greek letter *chi*.)

- Finding the chromatic number of a planar graph is equivalent to find the minimum number of colors required to color a planar map so that no two adjacent regions are assigned the same color.
- This question has been studied for more than 100 years.
- The answer is provided by one of the most famous theorems in mathematics.

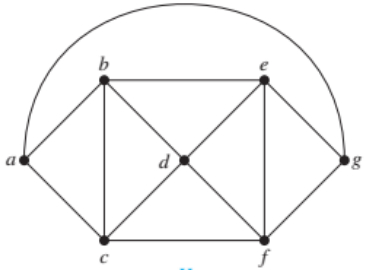
THE FOUR COLOR THEOREM The chromatic number of a planar graph is no greater than four.

Graph Coloring (*Contd...*)

- What are the chromatic numbers of the following graphs?

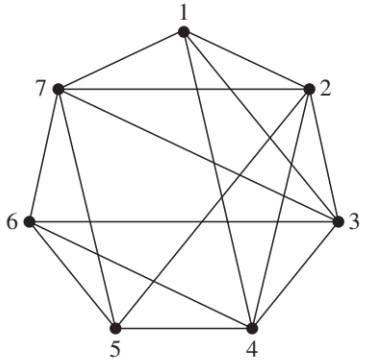


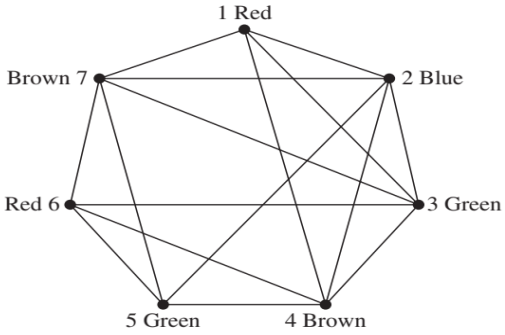
G



H

- What is the chromatic number of K_n?
- What is the chromatic number of the complete bipartite graph $K_{m,n}$, where m and n are positive integers?
- What is the chromatic number of the graph C_n, where $n \geq 3$? (Recall that C_n is the cycle with n vertices.)
- Applications of graph colorings
  - Graph coloring has a variety of applications to problems involving *scheduling* and *assignments*.





Time Period	Courses
I	1, 6
II	2
III	3, 5
IV	4, 7

/2026

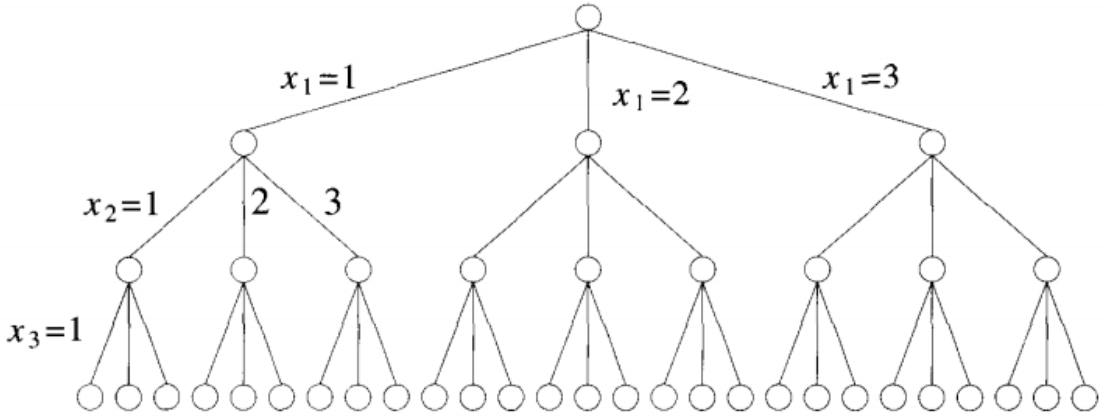
Graph Coloring (*Contd...*)

- A university has to schedule the final examinations for the following eight courses:CS101, CS102, CS201, CS202, CS301, CS302, CS401, CS402
- Two courses cannot be scheduled in the same time slot if there is at least one student enrolled in both courses.
- The following pairs of courses have common students: (CS101, CS102),(CS101, CS201),(CS101, CS301),(CS102, CS201),(CS102, CS202),(CS102, CS302),(CS201, CS202),(CS201, CS301),(CS201, CS401),(CS202, CS301),(CS202, CS402),(CS301, CS302),(CS301, CS401),(CS302, CS402),(CS401, CS402).
- The university wants to use the minimum possible number of examination time slots.
- *Hint: Represent each course by a vertex. Connect two vertices if at least one student is enrolled in both courses. Each color represents one examination time slot.*

Graph Coloring Algorithm

```
1   Algorithm mColoring(k)
2   // This algorithm was formed using the recursive backtracking
3   // schema. The graph is represented by its boolean adjacency
4   // matrix G[1 : n, 1 : n]. All assignments of 1, 2, ..., m to the
5   // vertices of the graph such that adjacent vertices are
6   // assigned distinct integers are printed. k is the index
7   // of the next vertex to color.
8   {
9       repeat
10      {// Generate all legal assignments for x[k].
11          NextValue(k); // Assign to x[k] a legal color.
12          if (x[k] = 0) then return; // No new color possible
13          if (k = n) then     // At most m colors have been
14                              // used to color the n vertices.
15              write (x[1 : n]);
16          else mColoring(k + 1);
17      } until (false);
18  }
```

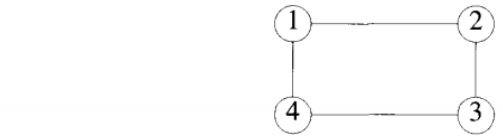
- Nonplanar graphs can have arbitrarily large chromatic number
- Two things are required to show that the chromatic number of a graph is *m*.
  - Show that the graph can be colored with *m* colors.
  - Show that the graph cannot be colored using fewer than *m* colors

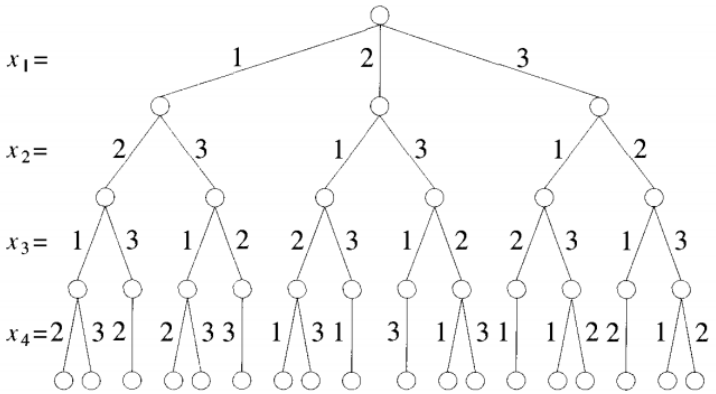


State space tree for mColoring when n = 3 and m = 3

Graph Coloring Algorithm

```
1   Algorithm NextValue(k)
2   // x[1], ..., x[k − 1] have been assigned integer values in
3   // the range [1, m] such that adjacent vertices have distinct
4   // integers. A value for x[k] is determined in the range
5   // [0, m]. x[k] is assigned the next highest numbered color
6   // while maintaining distinctness from the adjacent vertices
7   // of vertex k. If no such color exists, then x[k] is 0.
8   {
9       repeat
10      {
11          x[k] := (x[k] + 1) mod (m + 1); // Next highest color.
12          if (x[k] = 0) then return; // All colors have been used.
13          for j := 1 to n do
14          {   // Check if this color is
15              // distinct from adjacent colors.
16              if ((G[k, j] ≠ 0) and (x[k] = x[j]))
17              // If (k, j) is and edge and if adj.
18              // vertices have the same color.
19                  then break;
20          }
21          if (j = n + 1) then return; // New color found
22      } until (false); // Otherwise try to find another color.
23  }
```





A 4-node Graph and all possible 3-coloring

Module 4:
Advanced Graph Algorithms
(Part – 2)

Dr. Nirnay Ghosh

Planar Graphs

- Planar graph:

A graph is called *planar* if it can be drawn in the plane without any edges crossing (where a crossing of edges is the intersection of the lines or arcs representing them at a point other than their common endpoint). Such a drawing is called a *planar representation* of the graph.

- A graph may be planar even if it is usually drawn with crossings, because it may be possible to draw it in a different way without crossings.

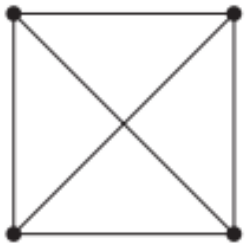


FIGURE 2 The Graph K_4.

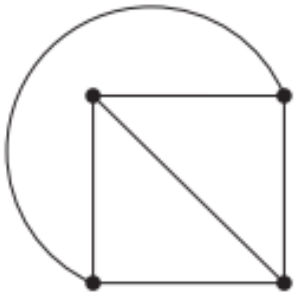


FIGURE 3 K_4 Drawn with No Crossings.

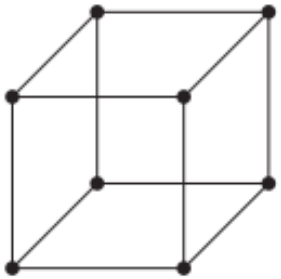


FIGURE 4 The Graph Q_3.

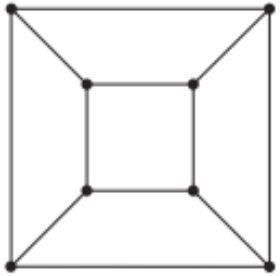
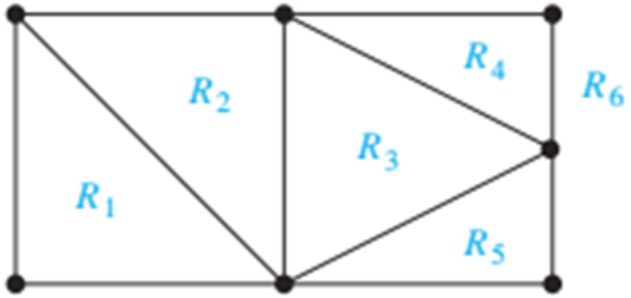


FIGURE 5 A Planar Representation of Q_3.

Planar Graphs

- We can show that a graph is planar by displaying a planar representation.
- A planar representation of a graph splits the plane into **regions**, including an unbounded region.
- Is $K_{3,3}$ planar?
- Applications of Planar graphs:
  - Design of electronic circuits: We can print a circuit on a single board with no connections crossing if the graph representing the circuit is planar.
  - Design of road network: Suppose we want to connect a group of cities by roads. We can build this road network without using underpasses or overpasses if the resulting graph is planar.



The regions of Planar Representation of a Graph

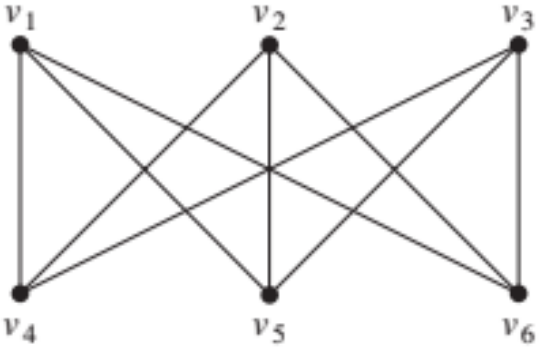
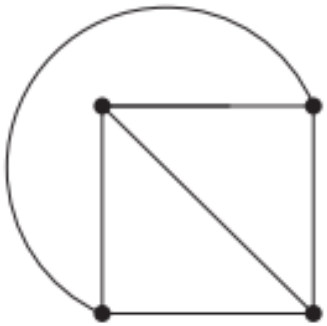
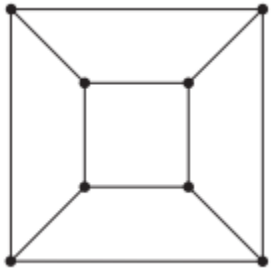


FIGURE 6 **The Graph $K_{3,3}$.**

Planar Graphs

- Degree of a region: Number of edges bordering a particular region.
  - *Degree of an interior region: number of edges enclosing the region*
  - *Degree of an exterior region: number of edges exposed to that region*
- When an edge occurs twice on the boundary (so that it is traced out twice when the boundary is traced out), it contributes two to the degree.
- We denote the degree of a region *R* by *deg(R)*.
- Find the regions and degree of each region for planar graphs given below.





Planar Graphs

EULER'S FORMULA Let G be a connected planar simple graph with e edges and v vertices. Let r be the number of regions in a planar representation of G. Then $r = e - v + 2$.

- Suppose that a connected planar simple graph has 20 vertices, each of degree 3. Into how many regions does a representation of this planar graph split the plane?
- Corollary-1:

If G is a connected planar simple graph with e edges and v vertices, where $v \geq 3$, then $e \leq 3v - 6$.

- Corollary-2:

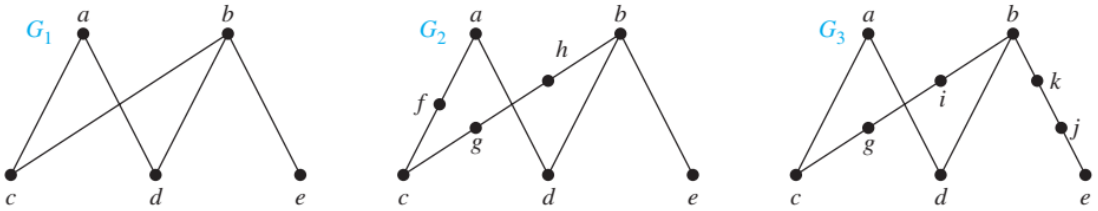
If G is a connected planar simple graph, then G has a vertex of degree not exceeding five.

- Corollary-3:

If a connected planar simple graph has e edges and v vertices with $v \geq 3$ and no circuits of length three, then $e \leq 2v - 4$.

Planar Graphs

- **Kuratowski's Theorem:**
  - $K_{3,3}$ and K_5 are not planar - a graph is not planar if it contains either of these two graphs as a subgraph.
  - All non-planar graphs must contain a subgraph that can be obtained from $K_{3,3}$ or K_5 using certain permitted operations.
  - If a graph is planar, so will be any graph obtained by removing an edge $\{u, v\}$ and adding a new vertex w together with edges $\{u, w\}$ and $\{w, v\}$. Such an operation is called an **elementary subdivision**.
  - The graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are called **homeomorphic** if they can be obtained from the same graph by a sequence of elementary subdivisions.

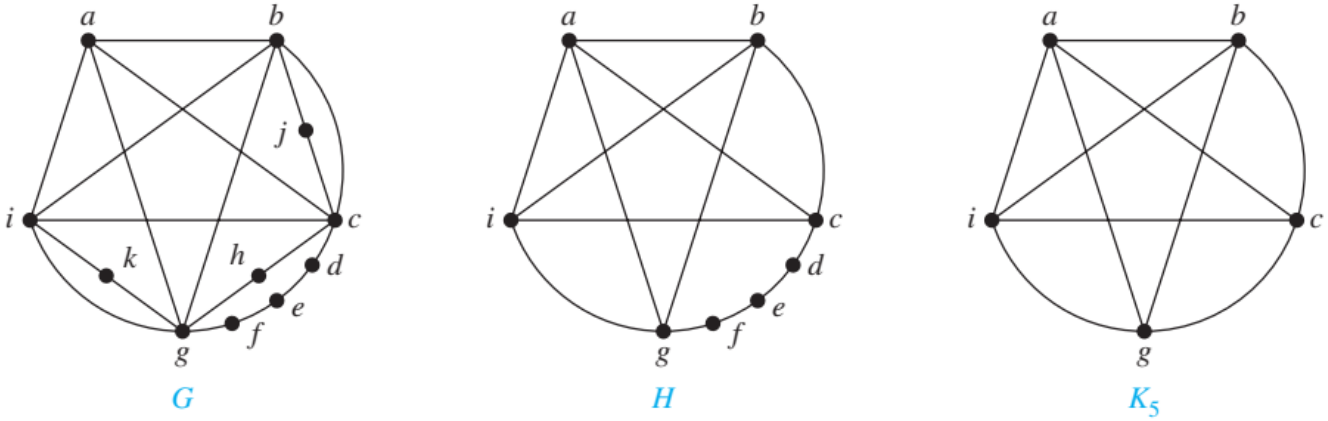


Homeomorphic Graphs

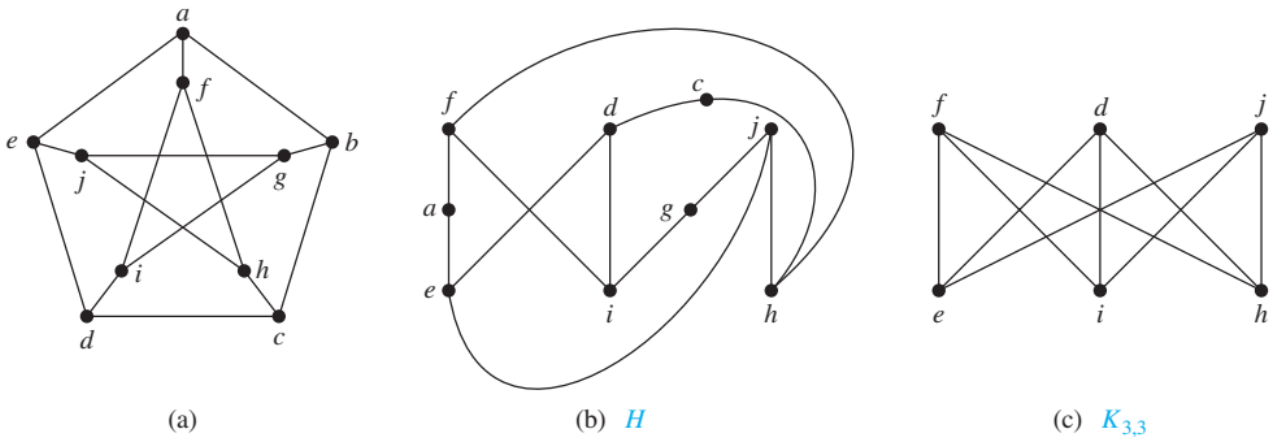
Theorem:

A graph is nonplanar if and only if it contains a subgraph homeomorphic to $K_{3,3}$ or K_5.

Planar Graphs



Undirected Graph G, a Subgraph H Homeomorphic to K_5.



The Petersen Graph, a Subgraph H Homeomorphic to $K_{3,3}$.

Module 4: Advanced Graph Algorithms (Part-3)

Dr. Nirnay Ghosh

Topological Sort/Ordering

- A topological sort of a DAG $G = (V, E)$ is a linear ordering of all its vertices such that if G contains an edge $\{u, v\}$, then u appears before v in the ordering.
  - Ordering of its vertices along a horizontal line so that all directed edges go from left to right.
  - Defined only in directed graphs that are acyclic

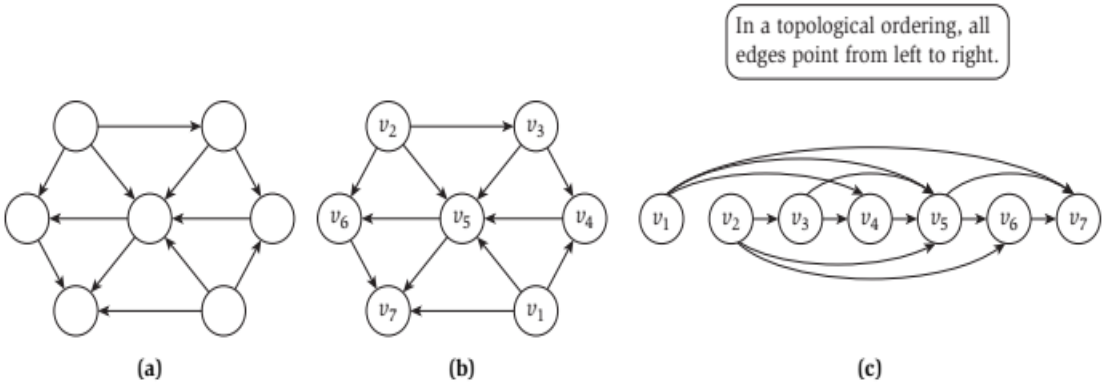


Figure 3.7 (a) A directed acyclic graph. (b) The same DAG with a topological ordering, specified by the labels on each node. (c) A different drawing of the same DAG, arranged so as to emphasize the topological ordering.

- DAGs can be used to encode *precedence relations* or *dependencies* in a natural way.
- We can represent such an interdependent set of tasks by introducing a node for each task, and a directed edge (i, j) whenever i must be done before j.
- If the precedence relation is to be at all meaningful, the resulting graph G must be a DAG. Indeed, if it contained a cycle C, there would be no way to do any of the tasks in C

(3.18) *If G has a topological ordering, then G is a DAG.*

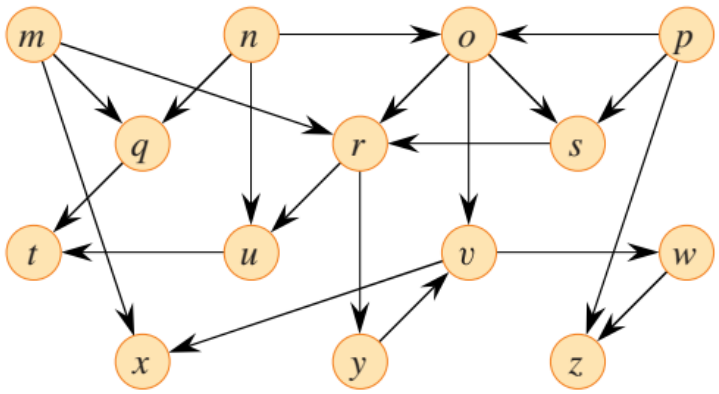
Topological Sort/Ordering

- Two methods to obtain topological sort/order
  - DFS
  - Vertex deletion method (Source-removal method)
- DFS Method
  - Step-1: traverse the graph G using the DFS method
  - Step-2: read the order in which the vertices become the dead end (i.e., the popping order of the nodes)
  - Step-3: reverse the popping order to get the topological order

```
TOPOLOGICAL-SORT(G)
1  call DFS(G) to compute finish times v.f for each vertex v
2  as each vertex is finished, insert it onto the front of a linked list
3  return the linked list of vertices
```

- Time Complexity:
  - The TOPOLOGICAL-SORT procedure runs in $\Theta(V+E)$ time
  - DFS takes $\Theta(V+E)$ time and it takes $O(1)$ time to insert each of the V vertices onto the front of the linked list.

Topological Sort/Ordering



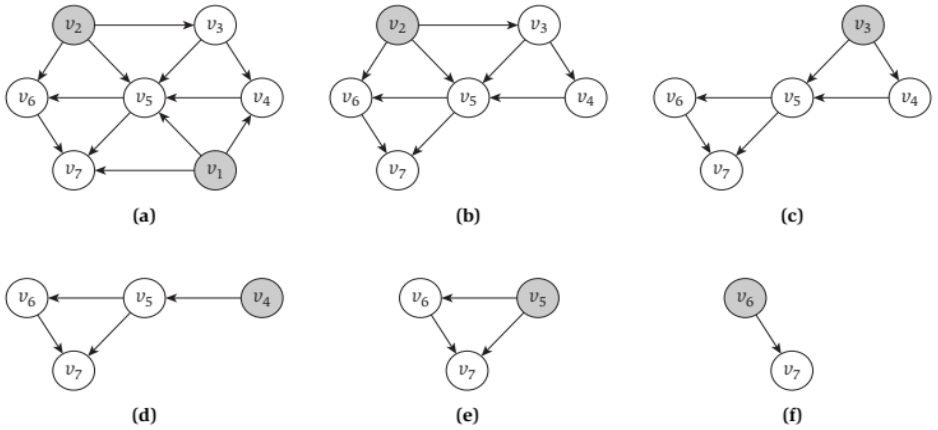


Figure 3.8 Starting from the graph in Figure 3.7, nodes are deleted one by one so as to be added to a topological ordering. The shaded nodes are those with no incoming edges; note that there is always at least one such edge at every stage of the algorithm's execution.

(3.19) *In every DAG G, there is a node v with no incoming edges.*

- Vertex deletion method (Source-removal method)
  - Step-1: read the in-degree of all vertices in G
  - Step-2: enqueue the vertex/vertices which has in-degree of 0
  - Step-3: delete the chosen vertex v from G to get $G' = G - \{v\}$
  - Step-4: decrement the in-degree of those vertices where there was an incoming edge from the vertex placed in Step-2
  - Step-5: repeat steps 2-4 on G' until all vertices are enqueued.

```
To compute a topological ordering of G:
Find a node v with no incoming edges and order it first
Delete v from G
Recursively compute a topological ordering of G−{v}
  and append this order after v
```